

Python Functions

What is a Function?

A function is a reusable block of code designed to perform a specific task. Functions help reduce code repetition, improve program organization, and make programs easier to understand and maintain.

```
def greet():  
    print("Hello Students")  
  
greet()  
greet()  
greet()
```

Here, def is used to define a function, greet is the function name, and greet() is the function call.

Creating a Function

A function is created using the def keyword followed by the function name and parentheses.

```
def welcome():  
    print("Welcome to Python")  
  
welcome()
```

Defining a function does not execute it. The function runs only when it is called.

Function Call

A function call executes the statements written inside the function. A function can be called multiple times.

```
def add():  
    print(10 + 20)  
  
add()  
add()  
add()
```

Output: 30 is displayed three times.

Why Do We Use Functions?

Functions provide code reusability, reduce duplication, improve program organization, simplify debugging, and make code easier to read.

Parameters

A parameter is a variable written in the function definition that receives a value when the function is called.

```
def greet(name):  
    print("Hello", name)  
  
greet("Aman")  
greet("Rahul")
```

In this example, name is a parameter, while "Aman" and "Rahul" are arguments.

Multiple Parameters

A function can accept more than one parameter.

```
def add(a, b):  
    print(a + b)  
  
add(10, 20)
```

Parameter and Argument

Parameters are variables defined in a function. Arguments are the actual values supplied during the function call.

```
def add(a, b):  
    return a + b  
  
add(10, 20)
```

Positional Arguments

In positional arguments, values are assigned to parameters according to their position.

```
def student(name, age):  
    print(name, age)  
  
student("Aman", 21)
```

Here name receives "Aman" and age receives 21. Changing the order of the arguments changes which parameter receives each value.

Keyword Arguments

Keyword arguments explicitly specify the parameter name. Their order does not matter.

```
def student(name, age):  
    print(name, age)  
  
student(age=21, name="Aman")
```

Default Parameters

A parameter can have a default value. The default value is used when no argument is supplied for that parameter.

```
def greet(name="Student"):  
    print("Hello", name)  
  
greet()  
greet("Aman")
```

Return Statement

The return statement sends a value from a function back to the code that called the function.

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

The returned value can be stored in a variable, used in an expression, or passed to another function.

print() vs return

print() displays a value on the screen. return sends a value back to the caller. This distinction is important when the result needs to be reused.

```
def add(a, b):  
    print(a + b)  
  
x = add(10, 20)  
print(x)
```

The first output is 30, but x receives None because the function does not return the result.

Returning Multiple Values

Python allows a function to return multiple values. They are commonly received using multiple variables.

```
def calculate(a, b):
    return a + b, a - b, a * b

x, y, z = calculate(10, 5)

print(x)
print(y)
print(z)
```

Python packs the returned values into a tuple before they are unpacked into x, y, and z.

Return Without a Value

A return statement without a value returns None.

```
def test():
    return

x = test()
print(x)
```

Function With No Parameter and No Return

```
def message():
    print("Hello")
```

```
message()
```

Function With Parameters but No Return

```
def add(a, b):
    print(a + b)
```

```
add(10, 20)
```

Function With Parameters and Return

```
def add(a, b):
    return a + b
```

```
result = add(10, 20)
print(result)
```

Scope of Variables

Scope refers to the part of a program where a variable can be accessed. The most important scopes for beginners are local scope and global scope.

Local Variable

A variable created inside a function is generally local to that function and cannot normally be accessed outside it.

```
def test():
    x = 10
    print(x)

test()
```

Global Variable

A variable created outside a function is a global variable. A function can read a global variable if it is accessible in that scope.

```
x = 10
```

```
def test():
    print(x)

test()
```

Local and Global Variables With the Same Name

```
x = 100
```

```
def test():
    x = 50
    print(x)
```

```
test()
print(x)
```

The x inside the function is local, while the x outside the function is global. They are separate variables.

global Keyword

The global keyword is used when a function needs to assign a new value to a global variable.

```
x = 10

def change():
    global x
    x = 50

change()
print(x)
```

Lambda Functions

A lambda function is a small anonymous function. It is useful for short operations where defining a full named function may be unnecessary.

```
square = lambda x: x * x

print(square(5))
```

Equivalent normal function:

Lambda With Multiple Parameters

```
add = lambda a, b: a + b
```

```
print(add(10, 20))
```

Lambda With Conditional Expression

```
check = lambda x: "Even" if x % 2 == 0 else "Odd"
```

```
print(check(10))
print(check(7))
```

Lambda With map()

map() applies a function to every element of an iterable.

```
numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x * x, numbers))

print(squares)
```

Lambda With filter()

filter() selects elements for which the supplied condition is true.

```
numbers = [1, 2, 3, 4, 5, 6]

even = list(filter(lambda x: x % 2 == 0, numbers))

print(even)
```

Recursion

Recursion occurs when a function calls itself. A recursive solution should have a condition that eventually stops the calls.

```
def count(n):
    if n == 0:
        return

    print(n)
    count(n - 1)

count(5)
```

Base Case and Recursive Case

A base case is the condition that stops recursion. The recursive case is the part where the function calls itself with a smaller or simpler problem. Without a proper base case, recursion can continue until Python raises a RecursionError.

Factorial Using Recursion

```
def factorial(n):
    if n == 0 or n == 1:
        return 1

    return n * factorial(n - 1)
```

```
print(factorial(5))
```

For example, factorial(5) is calculated as $5 \times 4 \times 3 \times 2 \times 1 = 120$.

Sum of Numbers Using Recursion

```
def sum_n(n):
    if n == 0:
        return 0

    return n + sum_n(n - 1)
```

```
print(sum_n(5))
```

Fibonacci Using Recursion

```
def fibonacci(n):
    if n <= 1:
        return n

    return fibonacci(n - 1) + fibonacci(n - 2)
```

```
print(fibonacci(6))
```

Recursion vs Loop

Loops repeat a block using iteration. Recursion repeats a task by making a function call itself. Recursion is especially useful for problems that naturally break into smaller instances of the same problem, such as tree traversal, backtracking, and divide-and-conquer algorithms.

Common Mistakes

Forgetting to call a function: defining a function alone does not execute it.

Confusing print() and return: print displays a value, while return sends a value back to the caller.

Missing arguments: a function requiring two parameters generally needs two corresponding arguments unless defaults are provided.

Recursion without a base case: recursive calls must eventually stop.

Accessing a local variable outside its function: local variables are limited to their local scope.

Quick Revision

def — defines a function

Function call — executes a function

Parameter — variable that receives an argument

Argument — actual value passed to a function

return — sends a value back to the caller

Local variable — variable available in its local scope

Global variable — variable defined outside functions

global — allows assignment to a global variable from inside a function

Lambda — small anonymous function

Recursion — a function calling itself

Base case — condition that stops recursion

Practice Questions

Basic

Create a function to print "Hello Python".

Create a function to print your name.

Create a function that accepts a number and prints its square.

Create a function that accepts two numbers and prints their sum.

Create a function to check whether a number is even or odd.

Create a function to find the largest of two numbers.

Create a function to find the largest of three numbers.

Create a function to calculate the area of a circle.

Create a function to calculate simple interest.

Create a function that accepts marks and prints a grade.

Return-Based

Return the square of a number.

Return the cube of a number.

Return the maximum of two numbers.

Return the maximum of three numbers.

Return whether a number is even or odd.

Return the factorial of a number.

Return the sum of digits of a number.

Return the reverse of a number.

Return whether a number is a palindrome.

Return whether a number is prime.

Scope

Demonstrate a local variable.

Demonstrate a global variable.

Create local and global variables having the same name.

Modify a global variable using global.

Write a program showing the difference between local and global scope.

Lambda

Create a lambda to find the square.

Create a lambda to find the cube.

Create a lambda to add two numbers.

Create a lambda to find the maximum of two numbers.

Create a lambda to check even and odd.

Use map() with lambda to calculate squares.

Use filter() with lambda to extract even numbers.

Use filter() with lambda to extract numbers greater than 50.

Recursion

Print numbers from 1 to N using recursion.

Print numbers from N to 1 using recursion.

Find factorial using recursion.

Find the sum from 1 to N using recursion.

Find the power of a number using recursion.

Find the nth Fibonacci number using recursion.

Reverse a string using recursion.

Check palindrome using recursion.

Find the sum of digits using recursion.

Find the GCD of two numbers using recursion.